

Bagaimana CNN (*Convolutional Neural Network*) bekerja?

Fajar Adie Santosa (@debug.stack)

2026

Contents

Pendahuluan	3
STEP 1: Input	3
STEP 2: Convolution dan ReLU	4
Convolution	4
ReLU	5
Image-to-Column	6
Conv Forward	8
STEP 3: Pooling	10
STEP 4: Flatten	11
STEP 5: Fully Connected	11
STEP 6: Softmax	13
STEP 7: Loss Function	13
STEP 8: Backpropagation	14
STEP 9: Gradien Descent	14
STEP 10 : Evaluasi Model	14

Pendahuluan

(TAMBAHKAN PRE-REQUISITE UNTUK BISA MEMAHAMI DOKUMEN INI : KALKULUS, ALJABAR LINEAR) Model CNN (*Convolutional Neural Network*) adalah salah satu jenis jaringan saraf tiruan yang digunakan untuk pengolahan data seperti gambar atau time series. Di era sekarang ini, membuat model CNN hanya memerlukan import library seperti TensorFlow atau PyTorch dan menulis beberapa baris kode untuk membangun model CNN. Namun, pernahkah berpikir, bagaimana sebenarnya CNN bekerja? mengapa memerlukan parameter dari library ini? dari rasa penasaran tersebut, dilakukan pembuatan ulang CNN dari awal menggunakan Python dan NumPy, tanpa menggunakan library deep learning apapun. Dengan begitu, dapat memahami bagaimana CNN bekerja.

STEP 1: Input

Pada proyek ini, input yang digunakan adalah gambar, menggunakan dataset MNIST (yang didapat dari kaggle). Terus, bagaimana Neural Network (NN) bisa memahami gambar? sepehaman saya, gambar merupakan kumpulan dari pixel, dan pixel merupakan kumpulan dari angka. Namun, angka dari gambar itu mempresentasikan apa? angka dari gambar itu mempresentasikan intensitas cahaya, dimana semakin besar angka, semakin terang pixel tersebut. Untuk lebih jelasnya, kalian bisa mempelajarinya di youtube ini. Menurut saya, penjelasan di youtube ini sangat mudah dipahami dan ada visual yang membantu untuk memahami konsep CNN ini.

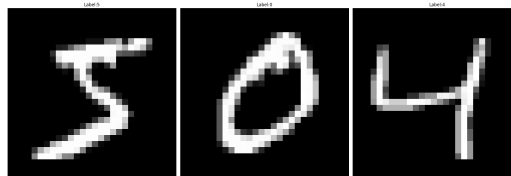


Figure 1: Contoh gambar digit dari dataset MNIST

Karena dataset didapat dari Yann LeCun, maka diperlukan library `idx2numpy` untuk mengubah dataset menjadi numpy array.

```
1 import idx2numpy
2 # for training
```

```

3 x_train=idx2numpy.convert_from_file("/kaggle/input/datasets/
   hojjatk/mnist-dataset/train-images.idx3-ubyte")
4 y_train=idx2numpy.convert_from_file("/kaggle/input/datasets/
   hojjatk/mnist-dataset/train-labels.idx1-ubyte")
5
6 # for test
7 x_test=idx2numpy.convert_from_file("/kaggle/input/datasets/
   hojjatk/mnist-dataset/t10k-images.idx3-ubyte")
8 y_test=idx2numpy.convert_from_file("/kaggle/input/datasets/
   hojjatk/mnist-dataset/t10k-labels.idx1-ubyte")

```

Listing 1: Contoh kode untuk mengubah dataset menjadi numpy array

intinya tambahkan juga untuk normalisasi data, karena data gambar memiliki nilai pixel 0-255, sehingga perlu dinormalisasi menjadi 0-1. Normalisasi ini dilakukan dengan membagi nilai pixel dengan 255.0. Dengan begitu, model CNN dapat lebih cepat dan lebih baik dalam belajar dari data gambar. Dengan rumus min-max normalisasi, yaitu:

$$x_{norm} = \frac{x}{255.0}, x \in [0, 1] \quad (1)$$

karena sangat mempengaruhi proses training.

STEP 2: Convolution dan ReLU

Convolution

Sebelum kita membahas lebih dalam tentang convolution, kita perlu memahami arti kernel. Kernel merupakan filter atau matriks kecil yang digunakan untuk mendeteksi fitur. Kernel ini akan melakukan proses konvolusi pada gambar input, dimana kernel akan menggeser posisi pada gambar input dan melakukan perkalian elemen demi elemen antara kernel dan bagian dari gambar input yang sedang diproses.

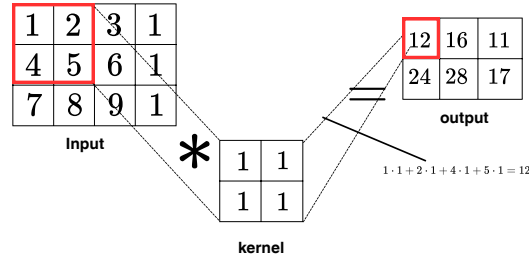


Figure 2: Ilustrasi proses konvolusi pada gambar input dengan kernel

Dari kernel yang bisa digeser posisinya, itu ada nama nya, yaitu stride. Stride merupakan berapa langkah pergeseran kernel pada gambar input. Semakin besar stride, semakin cepat proses konvolusi. Untuk rumus ukuran output dari proses konvolusi:

$$H_{\text{out}} = \frac{(H + 2 * \text{pad} - K_h)}{S} + 1 \quad (2)$$

yang dimana, K adalah panjang kernel, H adalah tinggi input (jumlah baris gambar/feature map).

$$W_{\text{out}} = \frac{(W_{\text{input}} + 2 * \text{pad} - K_w)}{S} + 1 \quad (3)$$

Kemudian, kita lanjut arti konvolusi. Konvolusi merupakan proses perkalian elemen demi elemen antara kernel dan bagian dari gambar input, kemudian hasil perkalian tersebut dijumlahkan dan ditambahkan bias, sehingga menghasilkan output. Output dari proses konvolusi ini disebut feature map. Untuk ilustrasi proses konvolusi bisa dilihat pada gambar 2.

ReLU

ReLU (*Rectified Linear Unit*) merupakan fungsi aktivasi yang digunakan untuk mengubah nilai negatif menjadi nol, dan nilai positif tetap.

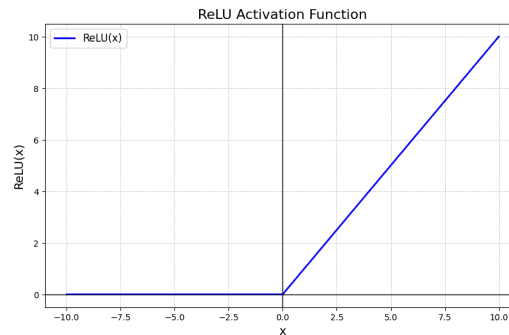


Figure 3: Relu Activation (sumber:geeksforgeeks)

Pada projek ini kita menggunakan ReLU sebagai fungsi aktivasi, karena berdasarkan penelitian (cari penelitiannya), ReLU lebih cocok untuk menyelesaikan masalah konvolusi. Aktivasi fungsi ReLU ini akan diterapkan pada hidden layer neural network.

Image-to-Column(im2col)

Kalau konvolusi diimplementasikan dengan for-loop biasa (geser kernel piksel demi piksel), prosesnya akan sangat lambat di Python/NumPy, karena NumPy dioptimasi untuk operasi matriks berukuran besar (perkalian matriks/vektorisasi), bukan untuk loop bertingkat. Trik yang umum dipakai adalah im2col (image-to-column): setiap patch gambar yang akan dikalikan dengan kernel diubah menjadi satu baris pada matriks besar, sehingga seluruh proses konvolusi bisa dihitung sebagai satu kali perkalian matriks saja

```

1 def im2col(x, kernel_size, stride=1):
2     """
3     Mengubah setiap patch gambar menjadi satu baris matrix.
4
5     Input:
6         x           : (N, C, H, W)
7         kernel_size : (KH, KW)
8         stride      : langkah pergeseran kernel
9
10    Output:
11        cols        : (N * H_out * W_out, C * KH * KW)
12    """
13
14    N, C, H, W = x.shape

```

```

15
16     KH, KW = kernel_size
17
18     # Ukuran output convolution
19     H_out = (H - KH) // stride + 1 # untuk menentukan ukuran
kernel pada height
20     W_out = (W - KW) // stride + 1 # untuk menentukan posisi
kernel pada weight
21
22     # Tempat menyimpan semua patch
23     cols = np.empty(
24         (N, H_out, W_out, C, KH, KW),
25         dtype=x.dtype
26     )
27
28     # Ambil setiap posisi kernel
29     for i in range(KH):
30         for j in range(KW):
31
32             cols[:, :, :, :, i, j] = (
33                 x[
34                     :,
35                     :,
36                     i:i + stride * H_out:stride,
37                     j:j + stride * W_out:stride
38                 ]
39                 .transpose(0, 2, 3, 1)
40             )
41
42     # Ubah semua patch menjadi baris
43     cols = cols.reshape(
44         N * H_out * W_out,
45         C * KH * KW
46     )
47
48     return cols

```

Listing 2: Fungsi im2col untuk mengubah patch gambar menjadi baris matriks

Setelah gambar diubah menjadi matriks *cols* berukuran $(N \cdot H_{\text{out}} \cdot W_{\text{out}}, C \cdot K_h \cdot K_w, F)$, dan kernel juga diratakan (reshape) menjadi matriks berukuran $(C \cdot K_h \cdot K_w, F)$ dengan F adalah jumlah filter, maka konvolusi cukup dihitung dengan satu kali perkalian matriks : $\text{cols} \odot W_{\text{col}}$

Convolution Forward

Dengan bantuan `im2col`, forward pass dari layer convolution bisa ditulis sebagai berikut: ubah input menjadi cols, kalikan dengan kernel yang sudah diratakan, tambahkan bias, lalu bentuk ulang hasilnya menjadi feature map

```
1  def conv2d_forward(x, W, b, stride=1):
2
3      N, C, H, W_input = x.shape
4      F, _, KH, KW = W.shape
5
6      # Padding
7      x_pad = x
8
9      # Ambil semua patch
10     cols = im2col(
11         x_pad,
12         kernel_size=(KH, KW),
13         stride=stride
14     )
15
16     # Ubah kernel menjadi matrix
17     W_col = W.reshape(F, -1).T
18
19     # Matrix multiplication
20     Z = cols @ W_col + b
21
22     pad=0
23
24     # Kembalikan ke bentuk feature map
25     H_out = (H + 2 * pad - KH) // stride + 1
26     W_out = (W_input + 2 * pad - KW) // stride + 1
27
28     Z = Z.reshape(
29         N,
30         H_out,
31         W_out,
32         F
33     )
34
35     Z = Z.transpose(0, 3, 1, 2) # (N, H_out, W_out, F)
36     transpose(0,3,1,2) -> (N, F, H_out, W_out)
37
38     conv_cache = (
39         x_pad,
```

```

40         stride,
41         cols,
42         x.shape
43     )
44
45     return Z, conv_cache
46
47 def maxpool_forward(x, size=2, stride=2):
48     """
49     x : input gambar
50
51     Max pooling forward. Nilai yang diambil hanya nilai yang
52     terbesar dari kernel
53     """
54     n, c, h, width = x.shape
55
56     h_out = (h - size) // stride + 1 # karena channel yang
57     dipakai hanya 1, berarti 1.1 = 1
58     w_out = (width - size) // stride + 1
59
60     out = np.empty((n, c, h_out, w_out), dtype=x.dtype)
61
62     for i in range(h_out):
63         for j in range(w_out):
64             h_start = i * stride
65             w_start = j * stride
66
67             window = x[
68                 :,
69                 :,
70                 h_start:h_start + size,
71                 w_start:w_start + size
72             ] # window ini hanya yang dimodif height dan
73             weight + size...
74
75             out[:, :, i, j] = window.max(axis=(2, 3)) # :, :
76             -> 1. Input Image, 2. Input Channel
77
78             # Simpan nilai yang dibutuhkan saat backward pass (posisi
79             window
80             # dipakai untuk menentukan ke mana gradient harus
81             dialirkan kembali)
82             pool_cache = (x, size, stride)
83
84     return out, pool_cache

```

```

79
80
81 def flatten_forward(x):
82     """
83     Ubah feature map 4D menjadi 2D untuk dense layer.
84     """
85     return x.reshape(x.shape[0], -1), x.shape

```

Listing 3: Forward pass layer convolution

Variabel `conv_cache` pada kode di atas menyimpan nilai-nilai yang dihasilkan saat forward pass (seperti input dan hasil `im2col`), yang nantinya dibutuhkan lagi saat menghitung gradien pada tahap backpropagation (lihat STEP 8), supaya tidak perlu dihitung ulang dari nol

STEP 3: Pooling

Pooling merupakan proses untuk mengurangi dimensi dari feature map/output (panjang dan lebar), namun menjaga informasi penting dari feature map tersebut. Formula untuk ukuran output dari proses pooling adalah

$$h_{\text{out}}, w_{\text{out}} = \frac{n_h - f}{s} + 1 \times n_c \quad (4)$$

yang dimana, n_h adalah tinggi input, n_w adalah lebar input, f adalah ukuran filter, s adalah panjang stride, dan n_c adalah jumlah channel. Untuk contoh, jika memiliki feature map dengan ukuran 4×4 , filter dengan ukuran 2×2 atau biasanya ditulis 2 dan stride 2×2 , maka:

$$\frac{4 - 2}{2} + 1 = 2 \quad (5)$$

output dari proses pooling adalah 2×2 , dengan channel yang sama dengan input. Pooling yang digunakan pada proyek ini adalah max pooling, yaitu mengambil nilai maksimum dari setiap filter. Untuk ilustrasi proses pooling bisa dilihat pada gambar berikut.

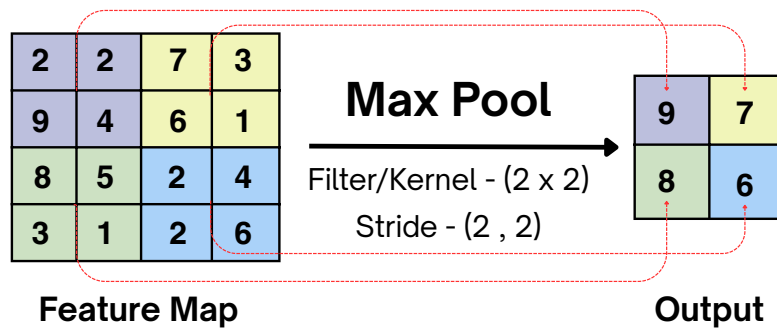


Figure 4: Ilustrasi proses max pooling pada feature map

Pooling biasanya disebut sebagai detektor fitur.

STEP 4: Flatten

Flatten merupakan proses untuk mengubah data yang memiliki dimensi besar direduksi. Misal, jika memiliki data dengan 2 dimensi, maka setelah proses flattening, data tersebut akan menjadi 1 dimensi. Proses flattening ini dilakukan agar data dapat diteruskan ke fully connected layer.

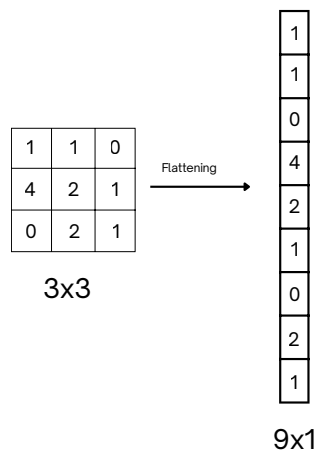


Figure 5: Ilustrasi proses flatten

STEP 5: Fully Connected

Fully connected atau dense layer merupakan kumpulan neuron yang saling terhubung dengan semua neuron pada layer sebelumnya. Satu neuron biasanya kita sebut sebagai perceptron. Perceptron ini memiliki bobot dan bias. Untuk ilustrasi perceptron bisa dilihat pada gambar dibawah ini.

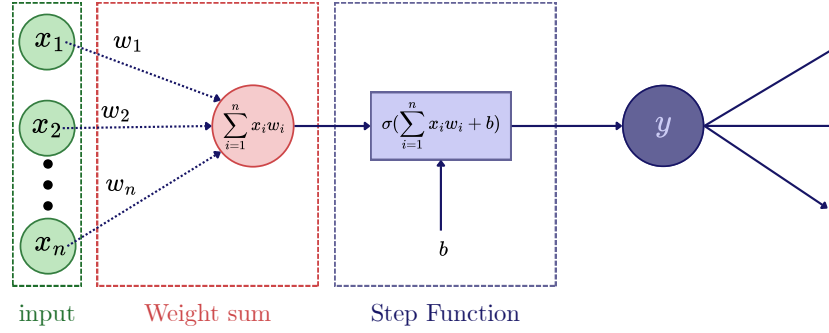


Figure 6: Ilustrasi perceptron

Dari proses perceptron ini, apabila kita memiliki banyak neuron, maka kita bisa menyebutnya sebagai fully connected layer. Secara rumus matematis, bisa ditulis sebagai:

$$A^{(l)} = \sigma^{(l)}(W^{(l)} A^{(l-1)} + b^{(l)}), l = 1, 2, \dots, L \quad (6)$$

dengan $A^{(0)} = X$ sebagai input, $W^{(l)}$ adalah matriks bobot pada lapisan ke- l , $b^{(l)}$ adalah vektor bias pada lapisan ke- l , $\sigma^{(l)}$ adalah fungsi aktivasi non-linear pada lapisan ke- l , dan L adalah jumlah total lapisan dalam jaringan.

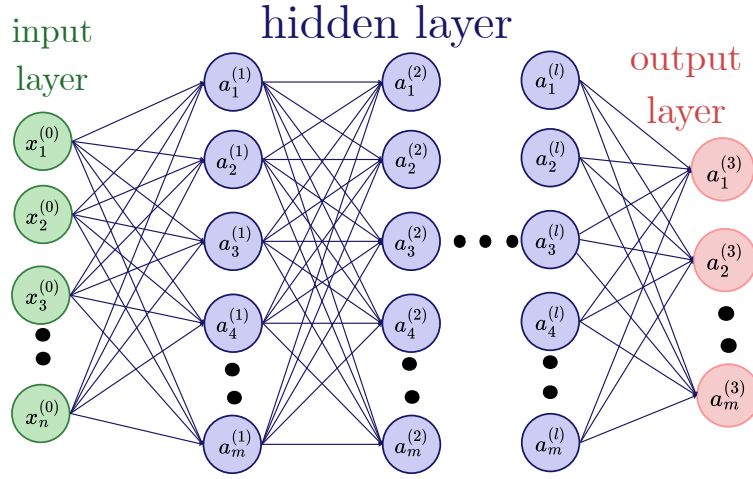


Figure 7: Ilustrasi arsitektur neural network dengan beberapa lapisan fully connected

STEP 6: Softmax

Softmax merupakan fungsi aktivasi yang mengubah ke distribusi probabilitas. Maksudnya adalah output dari softmax akan memiliki nilai antara 0 dan 1, dan jumlah dari semua output softmax adalah 1. Rumus dari softmax adalah:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad (7)$$

Pada proyek ini, softmax digunakan pada output layer fully connected layer, karena cocok digunakan untuk klasifikasi. Softmax ini akan mengubah model yang hasilnya masih raw menjadi probabilitas, sehingga dapat digunakan untuk menentukan kelas dari input data. Softmax ini seperti cross-entropy loss (kita bahas di step selanjutnya).

STEP 7: Loss Function

Loss function merupakan fungsi yang digunakan untuk mengukur seberapa baik model dalam memprediksi output. Pada proyek ini, loss function yang digunakan adalah cross-entropy loss, karena cocok digunakan untuk klasifikasi. Untuk rumus cross-entropy yang digunakan pada proyek ini adalah

versi binary, yaitu:

$$BCE = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)] \quad (8)$$

yang dimana:

- N adalah jumlah sampel
- y_i adalah true label dari sampel ke- i (0 atau 1)
- p_i adalah model-predicted probabilitas untuk class 1 untuk sampel i

STEP 8: Backpropagation

STEP 9: Gradient Descent

STEP 10: Evaluasi Model

(bikin epoch, batch size, dan learning rate sebagai hyperparameter, dan buat grafik loss function dan akurasi). Seberapa pengaruhnya dan tam

Kesimpulan

References

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. link.
- [2] Chitta Ranjan, S., & Saha, S. (2020). *Understanding Deep Learning Application in Rare Event Prediction*. Springer.